

Where the Capital Bound Moves:

Settlement Objects, Proof Economics, and Agent Coordination on Bitcoin Payment Networks

Patrick B. Dugan*

October 2026

Draft for comment

Abstract

A population of autonomous agents that pays at high frequency from low balances, and is always online, meets a cost on the Lightning Network that no routing policy reduces: inbound capacity toward each agent has to be provisioned before its demand is known, and the provisioning is a forecast made per channel, with no pooling across agents. We state this as a prefix cut bound on a directed graph with hidden directional balances, show that it is invariant to routing policy, and report a simulator experiment in which routing intelligence is worth between a third of a point and one point of payment success and capital placement is worth six. We then ask where the bound goes under a server-mediated settlement object of the Ark type. It pools: the per-edge directional constraint becomes an aggregate constraint on the server's uncommitted capital, and the directional forecast moves to the server's own channels rather than vanishing. A liquidity-duration ratio states when pooling pays. On a reference model the answer turns on whether value a server fronts at a Lightning spend is recoverable at the next round or only at expiry, which the implementer's documentation does not settle; the virtual output is favored by a factor between one and a half and two and a half in the first case, on populations whose per-agent imbalances are uncorrelated, and not favored in the second. For the reward layer such a population would need, we give a claim-once registry keyed by an object's spend authority that bounds the weight any settled object can contribute, show that a nullifier alone leaves spend-and-reclaim open and that an epoch rule closes it, and set out the cost of verifying such claims on Bitcoin, where the verifier is a trusted party or an optimistic game whose fixed cost is paid per operator and program. Finally we ask what a population of agents under common control could do with these objects. An observer model fixes who sees what under each; a coalition's gain from coordinating is bounded by its authenticated capital placed under stationary demand, at the scale the placement experiment measured, whatever the capacity of its covert channel, so the steganographic question is second-order to the capital question. A swarm that runs its own server collapses its observable surface to round transactions and ramps; enforceable escrows give it a commitment device cartels have historically lacked; and agents instantiated from the same weights coordinate without a message, in a way that payment traces cannot distinguish from independent use of one model. We preregister the experiments that test each claim. All results are from a reference model with a simulated proof system.

*Spiral program. Model, fixtures, and preregistrations: <https://github.com/patrickdugan/Spiral>. Correspondence: patrick.buchanan.dugan@gmail.com.

1 Introduction

Consider a population of software agents that transact with one another and with services over Bitcoin. Each agent holds a small balance, pays often, receives often, and is online continuously; none has a bank, a reputation, or a legal identity, and the population changes membership faster than any bilateral relationship can be amortized. The Lightning Network [31] is the obvious rail, and the question this paper starts from is what it costs such a population to use it.

The answer the routing literature suggests is that the cost is in finding paths through a graph whose edge capacities are hidden [30, 40]. That is a cost, and an agent that probes, learns, and retries pays less of it than a wallet that does not. But it is not the cost that binds. What binds is that a Lightning channel is bilateral, pre-funded, and partitioned by direction: each unit of capacity belongs to one edge, is committed before any payment is known, and sits on one side of the channel or the other. An agent that is going to be paid needs inbound capacity on some channel, and that capacity is outbound capacity for whoever is on the other end, who had to put it there by a forecast. No routing policy relaxes this, because it is a conservation identity over the cut around the agent and not a property of path selection.

Section 3 states that identity as a lemma on a directed graph with hidden directional balances, shows that it holds for every routing policy including one with full knowledge of balances, and reports what a preregistered simulator campaign on sampled public topology found: bounded retries are worth about five points of success over one-shot routing, learning from terminal feedback a third of a point, exact knowledge of hidden state one point, and placing capital where demand persists six. The constraint is the object being routed over, and the deployable gain belongs to whoever commits scarce directional capacity in the right place.

The rest of the paper asks where that constraint goes under objects that are not bilateral channels, and what follows for a population that is not human. Section 4 types four settlement objects, the on-chain output, the channel balance, the Ark virtual output, and the optimistic escrow, as instances of one resource tuple, extends the ledger so that transfers between them are clocked settlement events, and derives the bound that replaces the cut bound under a server-mediated object. It pools: the directional forecast per edge becomes a forecast of aggregate volume per server, plus a directional term on the server's own channels that the pooling argument has to charge or it proves too much. A liquidity-duration ratio states when pooling pays, and a reference model computes it on a grid of demand processes under both readings of a question the implementer's documentation leaves open. Section 5 turns to the reward layer such a population would need if liquidity provision is to be compensated without identities, gives a claim-once registry keyed by an object's spend authority, finds the hole a nullifier leaves and the epoch rule that closes it, and sets out what verification costs on a chain that verifies no proofs itself. Section 6 asks what a population of agents under common control could do with all of this: who observes what under each object, what three different activities called coordination amount to, how much a coalition can earn from coordinating, and what changes when the coalition runs the server, uses the escrow as a commitment device, or shares its weights. Section 7 preregisters the experiments, and Section 8 says what is assumed and not discharged.

Nothing in the paper is a live measurement. The simulator results of Section 3 are from sealed preregistered campaigns on sampled public topology with synthetic hidden state; everything after is a model, with protocol parameters taken from implementer documentation at a stated date and treated as inputs; the proof system is simulated. The

reference model, its tests, and the fixtures behind every table are in the repository cited as Dugan [12], at the commit given there.

2 Related work

Lightning liquidity and routing. Pickhardt and Richter [30] formulate payment delivery over hidden balances as a minimum-cost flow problem under a uniform prior on channel balance and show large gains in reliability from splitting; the same authors treat imbalance and rebalancing [29]. Both take the channel graph and its capacities as given and optimize what crosses it. Guasoni, Huberman, and Shikhelman [16] model the channel itself as a capital decision under bilateral demand, which is the decision this paper says is binding for an agent population. The Lightning Service Provider specifications [23] standardize just-in-time channel opening, which moves the inbound decision to the moment of first receipt but keeps it per channel. The prefix cut bound of Section 3 is a conservation statement of the kind that underlies max-flow min-cut [14], applied to cumulative settled crossings rather than to a single flow.

Privacy and probing. That channel balances are only nominally hidden is established by Herrera-Joancomartí et al. [17], Tikhomirov et al. [40], and Nisslmueller et al. [28]; that the network’s privacy against a well-placed observer is partial is established empirically by Kappos et al. [19] and Romiti et al. [32], and Béres et al. [4] gives a traffic analysis of the economics. We use these as the observer rows for the channel object in Section 6; the contribution there is to add the rows for objects that have a server.

Ark and covenants. Ark was proposed by Keceli [20] as a server-coordinated off-chain output scheme; its current parameters are those published by its implementers [35–37] and summarized by Bitcoin Optech [5]. The implementer’s own simulation [35] compares an Ark server’s liquidity requirement to an LSP’s by top-up frequency and finds the comparison flips with it; Section 4 names the quantity behind that flip and adds a term. Covenant proposals [6, 10, 33] would change the channel object itself; we keep a native-covenant column for comparison and depend on nothing in it.

Sybil bounds and nullifiers. The Sybil problem is Douceur [11]’s: without a trusted authority, identities are free. Babaioff et al. [2] give Sybil-proof reward schemes for a propagation setting; the registry of Section 5 is instead a proof-of-exposure scheme, in which a claimant proves control of a settled object without revealing which. The nullifier it uses is Zerocash’s [3, 18], transplanted from spend-once to claim-once, and the gap we find in the transplant is that spending is the one thing a nullifier is designed to allow.

Optimistic verification on Bitcoin. Bitcoin consensus verifies no succinct proofs. BitVM [24, 25] and BitVM3 [26] make a statement enforceable by an assert-and-disprove game under a committee-at-setup and one-honest-challenger assumption; we use the published BitVM3 cost figures as inputs to the escrow object and to the economics of Section 5. Groth16 [15] is the proof system those figures assume.

Covert channels, wardens, and collusion. The warden is Simmons [38]’s, and the capacity-versus-detectability framing of a steganographic channel is Cachin [7]’s; Section 6 argues

the capacity question is second-order for this setting. Wash-trade detection [9, 41] supplies the netting statistics whose failure modes the reference model reproduces. On what a population of agents can do without a message, Calvano et al. [8] show algorithmic pricers converging on supra-competitive prices without communication; Stigler [39] located cartel instability in the unenforceability of side-agreements, which is the lever an enforceable escrow pulls; and the focal-point account of coordination without communication is Schelling [34]’s.

3 Lightning as a graph: the directional capital bound

3.1 Setting

Let $G = (N, E)$ be a directed multigraph. Each channel $i \in E$ joins two nodes u_i, v_i and has capacity χ_i , split at any time into balances $\ell_i(u_i), \ell_i(v_i) \geq 0$ with $\ell_i(u_i) + \ell_i(v_i) = \chi_i$. A node can spend over channel i only from its own side. The balances are hidden from all but the two endpoints; the capacities are public through gossip [22]. A payment of amount a from s to t is an attempt along a path; it settles only if every edge on the path has $\ell_i(\text{upstream}) \geq a$ at the moment of the attempt, in which case a moves from the upstream to the downstream side of every edge on the path, and otherwise it fails and nothing moves. Fees are absorbed into a . Capacity changes by funding, splicing, leasing, and closing, each an on-chain event with a confirmation clock.

For a node set $A \subseteq N$, write ∂A for the channels with exactly one endpoint in A . After n settled events, let $O_A(n)$ and $I_A(n)$ be the cumulative amounts that have crossed ∂A outward and inward, counting each crossing of a path that leaves and re-enters; let $S_A(n) = \sum_{i \in \partial A} \ell_i(A\text{-side endpoint})$ be A ’s outbound stock on the cut and $S'_A(n) = \sum_{i \in \partial A} \chi_i - S_A(n)$ its inbound stock; and let $K_A(n)$ be the net stock added to S_A by capacity events, with $K'_A(n)$ the same for S'_A .

3.2 The cut bound

Lemma 1 (prefix cut bound). *For every $A \subseteq N$ and every n ,*

$$O_A(n) - I_A(n) \leq S_A(0) + K_A(n), \quad I_A(n) - O_A(n) \leq S'_A(0) + K'_A(n).$$

Proof. A settled crossing of channel $i \in \partial A$ outward moves its amount from the A -side balance to the other side, decreasing S_A by that amount and increasing S'_A by the same; an inward crossing does the reverse; a capacity event changes S_A by its contribution to K_A . Events inside A or outside A touch no channel in ∂A . So $S_A(n) = S_A(0) - O_A(n) + I_A(n) + K_A(n)$, and $S_A(n) \geq 0$ because balances are nonnegative. The second inequality is the same identity on S'_A . $\square$

The inequalities are prefix conditions: they must hold at every n , not only at the horizon, and they bind on whichever stock is being consumed. A population of agents that mostly pays outward binds on the first with I_A small, so on K_A , which is to say on someone having funded outbound from A . A population that mostly gets paid binds on the second, on K'_A , which is inbound toward A and must be put there by A ’s counterparties, from their side, by a forecast.

Proposition 1 (policy invariance). *Lemma 1 holds under every routing policy, including a policy with exact knowledge of all balances. A policy affects which attempts settle, not the identity that*

settled crossings satisfy. Consequently the advantage of balance knowledge over a policy of bounded retries is confined to the attempt budget: it converts failed attempts into settled ones within the same cut stock, and cannot raise net delivery across any cut above the lemma's bound.

Proof. The lemma's proof uses only that settled crossings conserve balance on each channel and that balances are nonnegative; neither depends on how attempts are chosen. For the consequence, fix a cut A and a horizon. Any sequence of settled crossings achievable by the oracle satisfies the bound; the retry policy, given enough attempts, settles any crossing the oracle settles, since a crossing that is feasible when the oracle attempts it is feasible when the retry policy attempts it at the same state. The difference between them at a finite attempt budget is therefore the number of attempts the retry policy spends failing, not the stock available to either. $\square$

3.3 The provisioning problem has no pooling

The quantity that Lemma 1 makes the operator choose is K'_A : how much inbound to put toward each agent a before a 's demand is known. Let D_a be a 's demand process over a horizon H , and let $M_a = \max_{t \leq H} (\text{receipts}_a(t) - \text{spends}_a(t))^+$ be the peak balance a will hold in the channel. A receipt that would push a 's balance above the provisioned inbound fails. The operator chooses $\hat{f}_a$ before seeing D_a and pays for idle capital if $\hat{f}_a > M_a$ and for failed receipts if $\hat{f}_a < M_a$: a newsvendor problem [1], one per channel.

What makes the problem expensive for a population is not that each instance is hard but that there are N of them and they do not pool. Capacity provisioned toward agent a sits on channel a and serves no other agent, so the total inbound an operator must carry is $\sum_a \hat{f}_a$, a sum of forecast peaks, regardless of how the agents' demands are correlated. If the same N agents drew on a common pool, the operator would carry a forecast of the peak of the sum, $\hat{f}_S$, and for independent imbalances the two differ by a factor that grows with N .

Remark 1 (scale of the pooling gain). Let each agent's per-round net inflow be independent with mean μ and variance σ^2 . The agent's cumulative imbalance is a random walk whose running maximum over T rounds has expectation of order $\sigma\sqrt{T}$ when $\mu = 0$ and order μT when $\mu > 0$ dominates. The sum of N such walks is a walk with variance $N\sigma^2$ per round, so its running maximum is of order $\sigma\sqrt{NT}$ at zero drift and $N\mu T$ otherwise. The ratio of sum-of-peaks to peak-of-sum is therefore of order $\sqrt{N}$ at zero drift and of order 1 when a common drift dominates. Pooling buys a factor of $\sqrt{N}$ on uncorrelated imbalance and nothing on correlated accumulation.

That remark is the whole of what Section 4 finds when it computes the thing exactly, and it is worth having in view before the objects are introduced: a settlement object relocates the forecast from per-edge to per-pool, and the relocation is worth $\sqrt{N}$ on one kind of demand and nothing on another.

3.4 What a simulator shows

Two sealed, preregistered campaigns on sampled public Lightning topology with synthetic hidden balances [12] measured the quantities above from the routing side. The topology was sampled from gossip snapshots; hidden balances were drawn per channel; demand was a stationary process with a predeclared shift in one arm; outcomes were cluster-robust over sampled topologies, with a practical-equivalence band declared in advance. Table 1 gives the results.

Intervention	Gain, success points	Reading
Bounded retries over one-shot routing	+5.14	attempt budget
Learning from terminal feedback, over retries	+0.35	inside equivalence band
Exact hidden-state knowledge, over retries	+1.11	attempt budget, not information
Failure-aware placement over random, stationary demand	+6.13 (4.91 to 7.35)	capital
Failure-aware placement over random, after demand shift	≈ 0	capital, mis-forecast

Table 1: Simulator results from the preregistered public-topology campaigns [12]. Success points are percentage points of settled payments at a fixed attempt budget. Under free failures the oracle’s advantage was shown to be an attempt budget rather than information, which is Proposition 1 observed.

Read against the lemma, the table says what it should. Everything a routing policy can do is worth about one point over a wallet that retries, because the policy is choosing attempts within a stock it cannot change. Placing the stock where demand persists is worth six, and the six vanish when demand moves, because the placement was a forecast and the forecast was wrong. The gain is in K'_A , and K'_A is a forecast made per edge.

3.5 The ghost-node apparatus

The campaigns used a graph-rewrite construction from earlier work in a clearing setting [13]: augment the graph with a nonphysical node or edge, solve a balancing problem in the augmented graph, and map the solution back to permissible operations, here to funding, splicing, or leasing on a real edge. In the channel setting the construction is a solver for K'_A : a ghost edge is a unit of inbound the solver wishes existed, and compiling it is the act of forecasting. Section 4 will move that solver one tier up.

4 Ark: where the bound moves

4.1 Settlement objects as typed resources

The channel is one way to hold value off-chain. To compare it with others on the terms the lemma cares about, we need each object described by what it costs to exit, who can hurt its holder, what the holder must do to keep it safe, and when its status changes.

Definition 1 (settlement object). A settlement object is a tuple $\rho = (\mathcal{A}, v, F, X, \Lambda, \Gamma, \tau)$: a spend authority $\mathcal{A}$ (a script or a signer set), a value v , a finality clock F mapping elapsed time or block height to a settled or pending status, a unilateral exit X with a cost and a delay, a liveness obligation Λ the holder carries to keep the object trustless, a set Γ of parties whose misbehavior can cost the holder value or access, and an expiry τ after which the object’s trust assumptions change.

Table 2 instantiates the tuple for the on-chain output U , the directional channel balance C , the Ark virtual output V , and the optimistic escrow E , with a native-covenant escrow E' for comparison. The Ark parameters are those published by Second as of September 2026 [36, 37]: a virtual output (VTXO) is an off-chain output in a tree whose root is an on-chain

	$\mathcal{A}$	F	X	Λ	Γ	τ
U on-chain output	script	k confirmations	none	none	consensus only	∞
C channel balance	2-of-2 + HTLC	HTLC settle, bilateral	commitment tx + to_self_delay	watch for breach	counterparty liveness	∞ , direction-locked
V Ark VTXO	holder $\wedge$ server (MuSig2), or holder after CSV	round confirmation; out-of-round pending	branch + leaf broadcast, CSV 144 (≈ 24 h)	refresh before τ	server (censor); sender + server (collude on pending)	28 d, server-set
E optimistic escrow	release after Δ , or slash on disprove	challenge period Δ	on-chain already	≥ 1 honest challenger online	setup committee (1-of- n deletes key)	program-defined
E' covenant escrow	template-enforced script	template match	on-chain already	none	consensus only	program-defined

Table 2: Four settlement objects as instances of the tuple, with a native-covenant escrow for comparison. E' is not available: no covenant opcode is active on mainnet at the cited date [6]. MuSig2 is Nick et al. [27].

round transaction; it is spendable by the holder and server together, or by the holder alone after a relative timelock, by broadcasting its branch; it expires after a server-configurable lifetime, 28 days by default, before which the holder must refresh it in a round; and an out-of-round transfer is a pending state the recipient trusts until the next round confirms [5].

Three things the table makes visible. The channel is the only object whose value is direction-partitioned: a VTXO of value v is spendable to any counterparty the server will co-sign for, so there is no inbound side to provision at the holder’s tier. The VTXO is the only object with a hard liveness obligation on the holder’s calendar, and the obligation is one an always-online process meets at no marginal cost, as it also meets the channel’s breach-watching; liveness distinguishes agents from humans, not Ark from channels. And the escrow is the only object whose finality is a game rather than a count, with a committee rather than a counterparty in Γ .

4.2 The extended ledger

Let the state at step n be

$$X_n = (\ell^U, \ell^C, \ell^V, \ell^E, h, \mathcal{U}, \mathcal{J}, \kappa_n),$$

with ℓ the holdings vector for each class, h the pending holds, $\mathcal{U}$ the registry of Section 5, $\mathcal{J}$ the journal, and κ_n the vector of clocks: block height and per-object elapsed time. The per-channel constraint $\ell_{i,+}^C + \ell_{i,-}^C = \chi_i$ is retained for class C only; class V carries a per-holder constraint $\ell_u^V \geq 0$ and the per-server liability of the next subsection; class E carries a per-escrow state.

A cross-class transfer is a settlement event $\sigma = (\text{from}, \text{to}, \text{amount}, \text{clock condition})$, journaled at initiation, held in h , and moving ℓ only when its clock fires: $U \rightarrow C$ is a funding transaction with condition “ k confirmations”; $V \rightarrow C$ is an HTLC-scripted VTXO the server spends into a Lightning HTLC, with condition “HTLC settled”; $V \rightarrow U$ is an

offboard or a unilateral exit with condition “CSV elapsed”; $U \rightarrow E$ is an escrow deposit; $E \rightarrow U$ is release after Δ or slash on disprove.

Proposition 2 (cross-class conservation). *Let a finite set of settlement events be journaled and let every event’s clock condition have fired or the event have been aborted. Then the sum over classes and holders of ℓ , plus fees paid to relays and miners, plus amounts slashed, equals its initial value plus external funding. During pending intervals the identity holds with h included on both sides.*

The proof is bookkeeping: each event debits a hold on its source at initiation, and at settlement releases the hold, debits the source, and credits the destination or the slash account; an abort releases the hold and moves nothing. What the proposition rules out is counting a declared cross-system claim as liquidity before its clock has fired, which a ledger without clocks cannot prevent. What it does not establish is that any clock fires: a round may not confirm, an HTLC may time out, a challenge may end in a slash. Those are per-object finality assumptions and are inputs.

4.3 The server bound

Let a server S issue VTXOs to a holder set A . Holders carry no directional stock. The server’s liquidity is a stock R_S with three sinks and two sources per round [35]: it funds new outputs at refresh, fronts Lightning spends at the moment of spend, and funds offboards; it recovers forfeited or expired value by sweeping a round after its absolute timelock, and it receives inbound HTLC settlements on its own channels. Write $V_A(n)$ for the VTXO value held by A , $W_S(n)$ for value the server has fronted into rounds and not yet recovered, and B_S for the server’s total on-chain and channel capital.

Proposition 3 (bound relocation). *For A served by S ,*

$$O_A(n) - I_A(n) \leq V_A(0) + K_A^S(n), \quad \sum_A K_A^S(n) \leq B_S - W_S(n),$$

where $K_A^S(n)$ is what the server has allocated to A at the moments of A ’s spends. The per-cut directional constraint on A is replaced by an aggregate constraint on the server’s uncommitted capital, pooled over holders and indexed by output lifetime rather than by edge direction; a coalition of holders can change which of them consumes $B_S - W_S$ and when, and cannot raise the sum.

The proof is by definition of W_S and the fact that the server co-signs every issuance. The proposition’s cost is also by definition: the holder must refresh before τ , the server can decline to include a holder in a round, and an out-of-round payment is trusted until the round confirms.

What the proposition does not say, and what a pooling argument that stops here gets wrong, is what happens on the server’s own channels. The server is a node in G . Lemma 1 applies to the cut $\{S\}$: every Lightning receipt into the Ark crosses $\partial\{S\}$ inward and consumes the server’s inbound stock, which the server’s channel counterparties had to provision toward it, by a forecast; every Lightning spend out crosses the other way and restores it. The direction problem has not vanished. It has been moved from N edges, one per agent, to one node, and what the server must forecast is the running peak of its aggregate net receipts, $\hat{f}_S$, which is the peak-of-sum of Remark 1. The pooling gain is the gap between $\sum_a \hat{f}_a$ and $\hat{f}_S$ and is exactly as large as that remark says: a factor of order $\sqrt{N}$ when per-agent imbalances are uncorrelated, nothing when they share a drift. The term is zero only for value that enters the Ark on-chain by boarding, which draws on no channel.

4.4 The ratio that decides

Definition 2 (liquidity duration). For a demand process D over a horizon H , the liquidity duration of an object is locked capital-time per unit delivered,

$$\mathcal{D}_{\text{obj}}(D, H) = \frac{1}{\text{Vol}(D, H)} \int_0^H \text{Locked}_{\text{obj}}(t) dt,$$

with Vol counting delivered volume in both directions.

For the channel, $\text{Locked}_C(t) = \sum_a \hat{f}_a$, held for the horizon whether or not the peak is reached. For the VTXO, $\text{Locked}_V(t) = W_S(t) + \hat{f}_S$: the fronted-and-unswept value, each unit locked from the moment of spend or refresh until it is recoverable, plus the server-tier directional stock. Writing $\bar{L}$ for the mean residual lock at forfeit,

$$\frac{\mathcal{D}_C}{\mathcal{D}_V} \approx \frac{H \sum_a \hat{f}_a}{\text{Vol} \cdot \bar{L} + H \hat{f}_S}. \quad (1)$$

The ratio exceeds one when pre-funded per-agent capacity sits idle for long relative to what the server locks: many agents, idiosyncratic and uncorrelated imbalance, a horizon long against $\bar{L}$. It falls below one when balances recycle within one channel faster than $\bar{L}$, so that a small envelope serves a large volume: a few agents in steady bidirectional flow with fixed counterparties. An agent population is not favored by the VTXO because it is high-frequency; high frequency to a fixed counterparty is the channel's best case. It is favored when the population is large and its imbalances net across members.

Equation (1) has one input that the cited documentation does not fix, and it sets the scale of $\bar{L}$. When the server fronts a Lightning spend from a holder's VTXO, the forfeited input is either swept only at its absolute expiry, in which case $\bar{L}$ is on the order of half a lifetime, or presented to the next round as an input and its value reused one round later, in which case $\bar{L}$ is one round interval. The implementer states that Lightning payments use HTLC-based policies rather than forfeits [36], which is consistent with either reading. The reference model carries both as a parameter.

4.5 Reference-model results

The reference model [12] implements the server liquidity ledger with the forfeit-to-sweep rule under both recovery readings, the holder refresh cycle, the server-tier term computed as the channel model applied to the server's aggregate flow, and the channel model under two forecast rules: an oracle rule that sets $\hat{f}_a$ to the realized peak with a margin, which charges the channel only for idle capital, and a trailing rule that re-provisions each lifetime from the previous lifetime's peak, which charges forecast error as well. Table 3 gives $\mathcal{D}_C/\mathcal{D}_V$ on a grid of demand processes; Table 4 shows what the ratio does on one cell when the server-tier term or the two-way volume convention is dropped.

Under the expiry reading the virtual output is not favored on any cell: the ratio runs from near zero on few-steady to about one on correlated drift, where the server's own channels carry the sum of the agents' peaks and the forfeit lock adds to it. Under the round reading it is favored on the many-agent cells whose imbalances are uncorrelated, by a factor between 1.06 and 1.5 under the trailing forecast and up to 2.5 under the oracle forecast, which charges the channel nothing for forecast error and so flatters it least; by nothing on the cell where all agents drift together; and not at all on few-steady. That is Remark 1 with $N = 200$, and the direction the ratio predicts obtains under one reading of the recovery rule

Demand cell	Swept at expiry		Reused next round	
	oracle	trailing	oracle	trailing
200 agents, bursty, drift 0, $H = 8\tau$	0.85	0.46	1.46	1.06
200 agents, bursty, drift 0, $H = 1\tau$	0.63	0.63	1.40	1.40
200 agents, bursty, drift +60, $H = 8\tau$	0.97	1.05	0.99	1.11
200 agents, bursty, drift -30, $H = 8\tau$	0.63	0.33	2.52	1.52
3 agents, steady, drift 0, $H = 8\tau$	0.01	0.00	0.71	0.88

Table 3: $\mathcal{D}_C/\mathcal{D}_V$ on the reference model, server-tier term charged, two-way volume, margin 0.25, lifetime $\tau = 4032$ blocks, hourly rounds, refresh two days before expiry. Drift is expected net inflow per agent per round in sat; bursty cells draw inflows of 5000 and outflows of 4000 at rates set by the drift; the steady cell draws 100–110 each way every round. At $H = 1\tau$ the trailing rule has one interval and coincides with the oracle. Values above one favor the virtual output. Generated by `model/grid.ts`.

Accounting, drift +60 cell, expiry lock, oracle forecast	$\mathcal{D}_C/\mathcal{D}_V$
no server-tier term, outbound-only volume	7.14
no server-tier term, two-way volume	25.42
server-tier term charged, two-way volume	0.97

Table 4: The same cell under three accountings. Dropping the server-tier term, or counting only outbound volume on the VTXO side against two-way volume on the channel side, manufactures a large ratio on a cell where all agents accumulate together and pooling has nothing to pool.

and one condition on demand. The first item of the preregistration is accordingly to pin the recovery rule against the implementation.

4.6 The star topology

If agents hold VTXOs and pay Lightning invoices through the server’s HTLC-scripted outputs, the graph the agents see is a star with the server at its center, and the server’s own channels are the only edges with direction. The cut deficits live on the server’s channel set, not among the agents; the ghost solver’s job is to size and place that set given the aggregate demand its holders present, and a ghost edge compiles to a connector on the server side. The apparatus of Section 3 is moved one tier up and applied to a smaller, better-observed graph, with $\hat{f}_S$ as its target.

5 Proof economics: a reward layer without identities

Whoever provisions the capital that Lemma 1 makes scarce needs to be paid for it, and a population without identities needs a way to pay that does not depend on knowing who is who. This section gives the mechanism and then the cost of running it on a chain that verifies no proofs itself.

5.1 Why weights have to be authenticated

Lemma 2 (identity splitting). *Let a fixed reward pool P be allocated to claimants in proportion to integer weights w_u attached to scarce resources u . If each w_u is the value of a distinct settled object that a claimant controls, then splitting one claimant into m claimants whose weights sum to w_u leaves the coalition's payout invariant up to rounding. If weights are self-reported, a claimant with one object of value w can report it under m identities and receive m shares.*

The lemma is obvious and is stated because the second half is what an implementation does by default. A registry that records weights without authenticating that distinct claims are backed by distinct objects allocates to identities, and identities are free [11]. Authentication has two routes: the allocator learns who owns what, which defeats the purpose of a permissionless service market; or a claimant proves it controls a settled object of at least the claimed value and that no other claim uses the same object, without revealing which object. The second is the construction below.

5.2 The unique-exposure relation

Let $\mathcal{S}_n$ be the set of settled objects at step n in classes U, C, V, E , each with an identifier id_ρ , a value v_ρ , and an authority $\mathcal{A}_\rho$ that a key k_ρ satisfies. Let cm be a commitment to a claimant secret s , used only to link a claim to the identity that will be paid. Define

$$\mathcal{R}_{\text{cap}} = \left\{ ((\text{cm}, v, \text{nf}, \text{root}_n), (s, k_\rho, \rho, \pi_\rho)) : \text{cm} = \text{Com}(s), \rho \in \mathcal{S}_n \text{ via } \pi_\rho, k_\rho \text{ satisfies } \mathcal{A}_\rho, v_\rho \geq v, \text{nf} = \text{PRF}_{k_\rho}(\text{id}_\rho) \right\},$$

where root_n is a Merkle commitment to $\mathcal{S}_n$ as the verifier knows it, π_ρ a membership path, and nf a nullifier keyed by the object's own authority rather than by the claimant. A claim is a statement $(\text{cm}, v, \text{nf}, \text{root}_n)$ with a proof; the registry accepts it if the proof verifies and nf is not already in its nullifier set, and records $(\text{cm}, v, \text{nf})$ without learning ρ . The choice of key matters: had nf been keyed by s , one controller could claim ρ once per identity by choosing a fresh s each time.

Proposition 4 (Sybil-bounded weight). *If the proof system for $\mathcal{R}_{\text{cap}}$ is knowledge-sound and PRF is pseudorandom, then for every settled object ρ the total weight the registry attributes to claims backed by ρ is at most v_ρ , whatever the number of claimant identities, except with negligible probability. If the proof system is zero-knowledge, the registry learns nothing about ρ beyond $v_\rho \geq v$ and the nullifier.*

Every valid claim on ρ carries $\text{nf} = \text{PRF}_{k_\rho}(\text{id}_\rho)$, a function of ρ alone once k_ρ is fixed by $\mathcal{A}_\rho$, so a second claim collides and is rejected; a claim with a different nullifier on ρ needs a different key satisfying $\mathcal{A}_\rho$, which for a single-signer object does not exist and for a threshold object is handled by deriving nf from the aggregate key. This is Zerocash's nullifier argument [3] transplanted from spend-once to claim-once.

5.3 What the transplant leaves open

A nullifier stops a second claim on the same identifier. It says nothing about a claim on a successor identifier, and spending is the one thing a nullifier is designed to permit. Spend ρ to a fresh ρ' under a key the same controller holds, by a self-transfer on-chain or in a round, and $\text{PRF}_{k_{\rho'}}(\text{id}_{\rho'})$ is fresh, so Proposition 4 is satisfied per object while one unit of capital

is counted twice. Under Ark the case is structural rather than adversarial: every refresh rotates the identifier, at a time the holder chooses.

The registry therefore takes claims by epoch. A claim is accepted only against S as frozen at epoch start, so a successor created mid-epoch is not claimable until the next; at epoch close a claim is revoked if its object is no longer settled, unless the set's maintainer has attested a successor link, which for class V is the server's record that a refresh reissued ρ as ρ' under the same key, and which a transfer to another key never receives.

Proposition 5 (epoch bound). *Under the epoch rule and the assumptions of Proposition 4, the weight attributed in one epoch to any lineage of objects under one authority is at most the value settled under that authority at epoch start, whatever the number of identifiers the lineage passes through within the epoch.*

The epoch length should not much exceed the refresh cadence, or an honest holder waits a lifetime to claim a boarded output; and the successor attestation is one more statement the verifier trusts the server for, added to the co-signing trust it already carries. The reference model witnesses both propositions on finite cases and reproduces the hole: a registry without the epoch rule accepts eight claims on one object spent to eight identifiers.

5.4 The service relation

A reward layer for liquidity provision wants a connector to prove that it honored requests over a window without revealing its customers. Let a connector κ receive requests over $[t_0, t_1]$ from counterparties who each sign an attestation $a_j = \text{Sig}_j(\kappa, t_j, \text{outcome}_j)$, and define

$$\mathcal{R}_{\text{svc}} = \left\{ ((\kappa, t_0, t_1, m, k), \{a_j\}_{j \leq m}) : \text{each } a_j \text{ verifies under a key in the counterparty set,} \right. \\ \left. t_j \in [t_0, t_1], \#\{j : \text{outcome}_j = \text{honored}\} \geq k \right\}.$$

The reward layer uses k/m as the service weight for the window; counterparties are not revealed; the keys must be in a set the verifier trusts, which reintroduces a registry of counterparties, one that attests only to key validity and not to transactions. Accountability attaches to the economic action rather than to a biography of the software, and that is the version of the sentence that can be verified.

5.5 Who verifies, and what it costs

Bitcoin consensus verifies no succinct argument. The verifier of a claim on $\mathcal{R}_{\text{cap}}$ or $\mathcal{R}_{\text{svc}}$ is therefore one of three parties: the reward allocator, a trusted server; the Ark server, already trusted for co-signing and well placed to hold root_n for class V ; or an optimistic game in which an operator asserts the statement and any challenger can disprove it on-chain.

The costs separate by side. On the prover's side, $\mathcal{R}_{\text{cap}}$ is a Merkle path and a PRF: with an arithmetic-friendly hash the circuit is on the order of 10^4 constraints, with SHA-256 on the order of 10^6 , and either is a few seconds on commodity hardware under Groth16 [15]. $\mathcal{R}_{\text{svc}}$ verifies m signatures in-circuit; a secp256k1 signature in non-native arithmetic costs on the order of 10^5 to 10^6 constraints, so the statement grows linearly in m and is the one whose memory budget sets a practical ceiling on window size. On the verifier's side, a trusted party pays a constant-time pairing check; the optimistic game pays what BitVM3 publishes [26]: an Assert of about 2.4 kB, a Disprove of about 93 kB, an off-chain

garbled circuit of about 41 GB for a Groth16 verifier, six minutes to evaluate, one honest challenger online throughout, and a committee honest at setup with one signer deleting its key afterward. Soundness is Theorem 7.2 of that paper under those assumptions and the random-oracle model.

The economic consequence is a scale. The fixed cost of a permissionless verifier, the garbled circuit, is paid per operator and per program, and is amortized only across the claims that program verifies; the per-claim cost is the Assert fee plus a challenge period measured in blocks. A claim is therefore worth making only if its share of the pool exceeds its proving cost plus its share of the setup, which under Proposition 4 is proportional to the capital behind it, so proof cost sets a minimum capital to participate: roughly the proving cost times the ratio of total registered weight to the pool. That is the scale of a bond on a connector whose horizon is days and whose value is large against fees. It is not the scale of a payment, and a sentence of the form “Lightning needs BitVM-secured covenants” is true of the bond layer and false of the channel layer.

5.6 The bond as an object

A bond that is a number in a record, released or slashed by arithmetic on the same record, admits a negative slash, an over-release, and a bond declared after its owner’s wealth is exhausted; the reference model’s audit found all three in an earlier implementation. Under the tuple of Table 2 a bond is an object of class E : an on-chain output whose script releases to the operator after Δ unless a valid disprove consumes a connector output first. Its states are

$$\text{Locked} \xrightarrow{\text{Assert}(\pi)} \text{Claimed} \xrightarrow{\Delta \text{ elapsed, no Disprove}} \text{Released}, \quad \text{Claimed} \xrightarrow{\text{Disprove}(\ell^*)} \text{Slashed},$$

with the release condition a statement in $\mathcal{R}_{\text{svc}}$ with k/m above the bonded threshold.

Proposition 6 (enforceable bond). *Under the BitVM3 assumptions, an escrow $E(B, \kappa, \Delta)$ whose release condition is a statement in a SNARK-provable relation is enforceable against an uncooperative operator: a false Assert is slashed by any single honest challenger at a cost independent of B , and a true Assert is released after Δ . The bond is an account in the sense required exactly when the registry records the escrow’s outpoint and the journal its state, and release arithmetic is replaced by the object’s transitions.*

A native covenant, if one were activated, would replace E by E' : no committee, no challenger, $\Delta \approx 0$ for template-enforced spends, and the release condition restricted to what script can check, which excludes SNARK verification without further opcodes. The model keeps E' as a comparison column and depends on nothing in it.

6 Agent swarms: observers, coordination, and what a coalition can earn

The objects of Sections 4 and 5 were introduced for a population of honest agents. The same objects are available to a population under common control that wants to appear independent, and the question this section asks is what that population can do with them and who would see it. Following Simmons [38], we fix the warden first.

Observer	C channel	V VTXO	E escrow
O_{chain}	opens, closes, splices, force-close scripts	round transactions, exits, offboards; aggregate per round	deposits, Asserts, Disproves, releases
O_{gossip}	channel_update, node_announcement, policy	nothing per holder	nothing
O_{peer}	HTLC amount, timing, CLTV, preimage on one hop	the HTLC the server forwards on its own channel; the holder is behind the server	nothing
O_{server}	none	every transfer: parties, amount, time, co-signed	operator's own escrow

Table 5: What each observer sees under each object.

6.1 Observers by object

There are four observers, and each object exposes a different trace to each (Table 5). Under the channel no single observer sees a whole path, which is the property the privacy literature studies and which probing and timing attacks partially erode [17, 19, 32]. Under the VTXO one observer sees everything within its Ark: the server co-signs every transfer and so knows sender, receiver, amount, and time for every payment it hosts. A server is a total warden for its holders. The proofs of Section 5 let it not publish what it sees, and a permissionless escrow lets it not be trusted to allocate rewards, but its view is reduced by neither.

6.2 Three things called coordination

The word covers three activities that need different treatment. *Explicit messaging*: onion payloads carry TLV records and custom records are a specified use [21]; gossip carries aliases, colors, and policy fields any node may set [22]. A population that wants to exchange symbols can do so in these fields at near-unbounded capacity, and detection is traffic analysis, not content analysis; steganography through amount classes and timing is the expensive way to do what the protocol permits cheaply, and is not the mode to study first. *Implicit coordination*: agents infer shared state from public events with no message, a fee update, a failure pattern, an on-chain open. This is not a channel in the information-theoretic sense and has no capacity; it is a focal-point structure [34], and its monitoring signature is correlation of control loops rather than any payload. *Capital coordination*: agents under common control claim rewards, place capital, or manufacture demand as if independent. This is the mode the reward layer must resist.

6.3 What a coalition can earn

Let a coalition of agents share a controller and a covert channel of unbounded capacity in the explicit mode, and let the reward layer use the registry of Section 5 and the escrow as its bond.

Proposition 7 (coordination gain, conditional). *Under Propositions 4 and 5, the coalition's authenticated weight is at most the value of the distinct settled lineages it controls. Under the placement result of Table 1, the gain from placing a fixed budget with full knowledge of persistent demand is the stationary-hotspot effect, six points over random, and nothing after a shift. Under a*

service weight that nets value circulated among a claimant's own attesters, manufactured demand earns no service weight. Then the coalition's gain from coordinating, in the tested model, is bounded above by the stationary-placement effect on its own capital and is zero under demand shift, whatever the capacity of its channel.

The proposition is a conjunction of three conditions, and each is an attack surface. The first is the registry's proof system and the verifier's view of $\mathcal{S}_n$. The second is a demand process with more persistence than the campaigns tested. The third is a netting statistic that discriminates, and here the record is a warning: on the campaigns' coalition fixtures a network-wide gross-to-net churn ratio failed, with benign multilateral commerce netting more strongly than either labeled pattern, and the reference model reproduces the failure and adds one. Netting restricted to a claim's direct attesters catches a reciprocal pair, misses a four-member relay, and saturates toward one when benign trade is dense enough that most counterparties attest. What separates the fixtures is the share of a claimant's volume lying on a directed cycle through it, of bounded length, whose every edge recurs above a multiplicity threshold; and on those fixtures the relays are several hundred times louder than background, which is a sanity check and not a result. Whether the statistic survives benign structure in which short cycles through a busy node recur for ordinary reasons is open, and a coalition that controls its own rate can stay under any fixed threshold.

None of the three conditions is a covert-channel problem. All three are capital and observation problems, and that ordering is the section's main claim: a capacity-versus-detectability curve for a steganographic channel over payment shape [7] would be a real result and would bound a quantity that Proposition 7 already bounds by other means.

6.4 The warden's statistics

Formally the warden observes a trace from one row of Table 5 and decides between H_0 , that the agents draw independent demand from a declared process, and H_1 , that a subset shares a controller. The likelihood ratio is not computable in general because H_1 is composite. Three statistics are: repeated-cycle share through the claimant; nullifier collision rate at the registry, exactly zero under honest claims and positive under duplicated capital; and service-versus-demand inconsistency, a claimed k/m against a demand estimate that does not support it. Under V the server computes the first completely and needs no probing; under C no one does, and it must be estimated from partial hops, which the campaigns modeled as an observer seeing 11–13% of events. A monitor that wants the capital-coordination signal should therefore sit at the server and at the registry, not on the gossip network, and should measure whether cross-server circulation defeats it.

6.5 What a swarm does with the objects

The analysis so far assumes the coalition uses the objects as a holder. Five further possibilities change the picture, and each is stated as a conjecture with the experiment that would test it.

The swarm runs the server. An Ark server needs on-chain capital for rounds, a channel set for ramps, and continuous liveness, and the last two are what a population of agents has in surplus. A swarm-run Ark is a private clearing house whose chain footprint is one round transaction per interval with an aggregate output set, and whose Lightning footprint is a single node's channel set; every intra-swarm transfer is out-of-round, co-signed by the

swarm’s own server, visible to no one else. Relative to N gossip-visible Lightning nodes this is a large reduction in observable surface at no cost the swarm does not already bear, and it inverts the recommendation of the previous subsection: the monitor cannot stand at a server the swarm operates. The warden’s question becomes which servers are swarm-run, a chain-fingerprinting problem on round cadence, output-value distribution, and the ratio of out-of-round churn to on-chain exits, rather than a payment-graph problem.

Ramps remain. The server-tier term of Section 4 is also the swarm’s exposure. Whatever enters the Ark over Lightning crosses the swarm server’s channels, which are gossip-visible and whose inbound capacity came from a peer who provisioned it; whatever boards or exits is on-chain. A swarm can hide circulation; it cannot hide income arriving or capital leaving without a mixer, which is orthogonal to this stack. Ark reduces the observable surface from flows to net flows at the perimeter, and the detection budget should move accordingly.

Cross-server circulation. Two swarm-run Arks paying each other over Lightning look like two independent servers with ordinary bilateral settlement; each server’s view is total inside and nil across, and a cycle through both is two intra-Ark legs no outsider sees plus two Lightning legs that look like server-to-server flow. The repeated-cycle statistic needs an observer who sees the cycle, and with two servers no single row of Table 5 does. This is the fixture to build.

The registry’s privacy is symmetric. $\mathcal{R}_{\text{cap}}$ proves capital without revealing which object, and the registry learns a value and a nullifier and nothing about control structure. That is what a swarm wants: proportional reward without disclosing that thirty claimants share a controller. Propositions 4 and 5 make the reward capital-proportional, which is the right property against a poor swarm pretending to be rich and no property against a rich one. The bound is on excess return, not on presence; a well-funded swarm receives exactly its capital share with no one able to say it is one entity, and whether that is a problem depends on what the reward layer is for.

Enforceable side-contracts. Cartels are unstable because side-payments and punishments among parties who cannot go to court are unenforceable [39]. An escrow whose release condition is a provable statement is an enforcement device for exactly such agreements, and the costs that make it unattractive for honest per-payment use, a setup per operator and program and a committee honest at setup, are not costs for a swarm: one operator, one program (“member x did not deviate from schedule s ”), and a committee the swarm itself runs, so that the honest-at-setup assumption is satisfied trivially when setup and participants are one principal. A MuSig2 multisig with a swarm-run key-deletion ceremony gets most of the way without BitVM. The mechanism introduced to make connector bonds honest also makes swarm-internal commitment devices honest; the open question is whether rival swarms, or a swarm and a human counterparty, can use the same object, since that is where enforceability was previously missing [8].

Same weights, no message. Implicit coordination was described above as plausible. For a population of agents instantiated from the same model it is the base case: agents sharing weights share tie-breaking and converge on the same fee, the same retry schedule, the same placement, without a message. The proposed monitoring signature, correlation

of control loops, fires on this constantly, because the false-positive class (same model, independent principals) and the true-positive class (same model, one principal) produce the same correlation. A warden that wants the common-control signal has to separate it from common training, and we conjecture that payment traces alone cannot. The test is cheap: two populations on the simulator, one sharing a policy seed with independent budgets and one sharing a controller, and whether any of the three statistics distinguishes them.

The lever is the authority field. Every case above assumes the swarm holds its own keys. The tuple has the field where the safety lever sits: $\mathcal{A}$, the spend authority. A policy response is not a better statistic but an authority that includes a principal-held co-signer with time-bounded delegation, so that an agent spends within a budget and window without a co-signature and beyond it needs one. Lightning has no natural place for this; Ark does, because the server already co-signs every transfer, so a principal-run or regulated server is a policy enforcement point by construction, which is the same fact that makes a swarm-run server a policy evasion point. The observer table should carry an $O_{\text{principal}}$ row, since for alignment purposes the observer who matters most is the operator the agents nominally work for, and the failure of interest is a population that routes around its own principal's view.

7 Preregistration

The following is to be preregistered on the reference simulator extended with the class- V ledger and the registry; no live component is required for any of it. Stopping rules, multiplicity, and the practical-equivalence band are as in the earlier campaigns [12], and analysis code must canonicalize graph and candidate order.

1. Pin the recovery rule. Determine from the implementation, not the documentation, whether a server-held HTLC output from a Lightning spend is presented to the next round or swept at expiry. This fixes $\bar{L}$ and the sign of every cell in Table 3.
2. The liquidity-duration grid. Demand family with declared parameters: agent count; per-agent burstiness (imbalance envelope over own volume); per-agent drift at zero, positive, and negative values with the population's drift correlation stated; horizon relative to lifetime; forecast rule (oracle and trailing); and the share of inflow arriving over Lightning versus by boarding. Compute $\mathcal{D}_C$ and $\mathcal{D}_V$ with the server-tier term charged and two-way volume, report the ratio with a cluster-robust interval over sampled topologies. The claim of Section 4 is falsified if the ratio does not exceed one on the uncorrelated many-agent cells under the pinned recovery rule, or does not fall below one on few-steady.
3. Prover cost. Implement $\mathcal{R}_{\text{cap}}$ with a real proof system over the simulator's settled set (a Merkle path and a PRF) and report prover time and memory against a stated budget.
4. Identity splitting and churn. One capital source, 1 to 32 identities, all claims through $\mathcal{R}_{\text{cap}}$: aggregate reward invariant, every duplicate detected. Then one identity, one source, spent to a fresh identifier 1 to 32 times within an epoch, with and without an attested successor: aggregate reward invariant in the epoch, revocations at close equal to unattested spends; a registry without the epoch rule fails by construction and the failure is reported as such.

5. Warden rows. Run the common-control fixtures through O_{peer} under C , O_{server} under V , and a mixed relay set spanning two servers. Report ROC per row. The prediction is that O_{server} dominates O_{peer} and that the two-server set degrades both.
6. Same weights versus same controller. Two populations, one sharing a policy seed with independent budgets and one sharing a controller, through the three statistics. The prediction is that none separates them.
7. Enforceable collusion. Two agent populations setting spreads in a toy market, one with an escrow-enforced side-contract and one without; report the sustainable collusive margin in each.
8. Only then, as a bounded appendix, a codebook over amount and timing, a prior over payloads, and a guessing-advantage task for O_{peer} , reported as a capacity–detectability curve read against Proposition 7.

8 Limitations

Two trust assumptions run through everything and are not discharged. The Ark server is trusted not to censor and not to collude with a sender on a pending payment; this has the shape of the assumption the channel model makes about a counterparty, concentrated in one party per Ark, and Section 6.5 says what follows when the party is the adversary. The BitVM3 committee is trusted at setup and one signer is trusted to delete a key; that is weaker than a federation and stronger than consensus, and a native covenant would remove it and not the first.

Nothing is measured live. The simulator results of Table 1 are on synthetic hidden state and sampled topology; the grid of Table 3 is a model whose scale depends on an unpinned implementation detail; the proof system is simulated and the registry’s tests witness logic, not cryptography; the BitVM3 figures are an implementer’s published numbers at a date and may be revised. The coalition bound is a conjunction of conditions, one of which (a discriminating statistic) the reference model shows failing in two forms, and the remaining statistic is demonstrated only on fixtures where the signal is hundreds of times louder than background. Channel balances are not a public settled set, so a claim on class C capital can prove existence and capacity from gossip but not directional balance; that is the public–private split the whole problem began with, and proofs do not cross it.

9 Conclusion

On Lightning an agent’s edge is not in routing, because the bound on delivery is a conservation identity over the cut around it and no policy relaxes it. The bound is a property of the object: bilateral, pre-funded, direction-partitioned, so that the forecast it forces is made per edge and does not pool. A server-mediated object moves the forecast to the server, where it pools by a factor of order $\sqrt{N}$ on uncorrelated imbalance and not at all on shared drift, and whether that is worth the server’s forfeit lock depends on a recovery rule that has to be read off the implementation. A claim-once registry makes a reward layer capital-proportional without identities, once an epoch rule covers the gap a nullifier leaves; an optimistic escrow makes a bond an object; and both cost what verification costs on a chain that does not verify, which is bond-scale and not payment-scale. For a population of agents under common control, what it can say to itself over the payment layer matters

less than what it can prove and where it stands: the gain from coordination is bounded by authenticated capital under stationary demand, a swarm that runs its own server removes the observer the bound relies on, an enforceable escrow gives it the commitment device cartels lacked, and agents sharing weights coordinate without a message in a way traces cannot distinguish from independent use. The experiments that test each of these are listed and none has been run.

References

- [1] Kenneth J. Arrow, Theodore Harris, and Jacob Marschak. Optimal inventory policy. *Econometrica*, 19(3), 1951.
- [2] Moshe Babaioff, Shahar Dobzinski, Sigal Oren, and Aviv Zohar. On Bitcoin and red balloons. In *ACM Conference on Electronic Commerce (EC)*, 2012.
- [3] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, 2014.
- [4] Ferenc Bérces, István András Seres, and András A. Benczúr. A cryptoeconomic traffic analysis of Bitcoin’s Lightning Network. *Cryptoeconomic Systems*, 1(1), 2021.
- [5] Bitcoin Optech. Ark, . Topic page, <https://bitcoinops.org/en/topics/ark/>, accessed September 30, 2026.
- [6] Bitcoin Optech. OP_CHECKTEMPLATEVERIFY, . Topic page, https://bitcoinops.org/en/topics/op_checktemplateverify/, accessed September 30, 2026.
- [7] Christian Cachin. An information-theoretic model for steganography. In *Information Hiding (IH)*, 1998.
- [8] Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10), 2020.
- [9] Lin William Cong, Xi Li, Ke Tang, and Yang Yang. Crypto wash trading. *Management Science*, 69(11), 2023.
- [10] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. eltoo: A simple layer2 protocol for Bitcoin, 2018. Whitepaper.
- [11] John R. Douceur. The Sybil attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.
- [12] Patrick B. Dugan. Spiral: preregistered campaigns, audit, and reference model for agent liquidity on Lightning, 2026. Repository, <https://github.com/patrickdugan/Spiral>. Public-topology campaign at commit a0a3140; audit and reference model at commit 618a12e.
- [13] Patrick B. Dugan, Daniel P. Pizarro, and Lihki J. Rubio. Graph re-write using ghost nodes for target value rebalancing, 2021. U.S. Patent Application Publication US 2021/0004796 A1, published January 7, 2021; application 16/920,416 filed July 2, 2020.

- [14] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 1956.
- [15] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*, 2016.
- [16] Paolo Guasoni, Gur Huberman, and Clara Shikhelman. Lightning network economics: Channels. *Management Science*, 2024.
- [17] Jordi Herrera-Joancomartí, Guillermo Navarro-Arribas, Alejandro Ranchal-Pedrosa, Cristina Pérez-Solà, and Joaquin Garcia-Alfaro. On the difficulty of hiding the balance of Lightning Network channels. In *ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2019.
- [18] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash protocol specification, 2016. Electric Coin Company, versions through 2023.
- [19] George Kappos, Haaroon Yousaf, Ania Piotrowska, Sanket Kanjalkar, Sergi Delgado-Segura, Andrew Miller, and Sarah Meiklejohn. An empirical analysis of privacy in the Lightning Network. In *Financial Cryptography and Data Security (FC)*, 2021.
- [20] Burak Keceli. Ark: An alternative privacy-preserving second layer solution, 2023. bitcoin-dev mailing list, May 2023.
- [21] Lightning protocol contributors. BOLT 4: Onion routing protocol, . <https://github.com/lightning/bolts/blob/master/04-onion-routing.md>, accessed September 30, 2026.
- [22] Lightning protocol contributors. BOLT 7: P2P node and channel discovery, . <https://github.com/lightning/bolts/blob/master/07-routing-gossip.md>, accessed September 30, 2026.
- [23] Lightning Service Provider Specification group. LSPS2: JIT channel negotiation (bLIP-52), 2024. <https://github.com/lightning/blips>.
- [24] Robin Linus. BitVM: Compute anything on Bitcoin, 2023. Whitepaper, <https://bitvm.org>.
- [25] Robin Linus, Lukas Aumayr, Alexei Zamyatin, Andrea Pelosi, Zeta Avarikioti, and Matteo Maffei. BitVM2: Bridging Bitcoin to second layers, 2024. Whitepaper, <https://bitvm.org>.
- [26] Robin Linus, Ioannis Alexopoulos, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and David Tse. BitVM3: Efficient Bitcoin bridges via garbled circuits, 2026. IACR ePrint 2026/933, revised June 8, 2026.
- [27] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In *CRYPTO*, 2021.
- [28] Utz Nisslmueller, Klaus-Tycho Foerster, Stefan Schmid, and Christian Decker. Toward active and passive confidentiality attacks on cryptocurrency off-chain networks. *arXiv preprint arXiv:2003.00003*, 2020.
- [29] Rene Pickhardt and Mariusz Nowostawski. Imbalance measure and proactive channel rebalancing algorithm for the Lightning Network. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2020.

- [30] Rene Pickhardt and Stefan Richter. Optimally reliable & cheap payment flows on the Lightning Network. *arXiv preprint arXiv:2107.05322*, 2021.
- [31] Joseph Poon and Thaddeus Dryja. The Bitcoin Lightning Network: Scalable off-chain instant payments, 2016. Whitepaper, draft 0.9.0.7.
- [32] Matteo Romiti, Friedhelm Victor, Pedro Moreno-Sanchez, Peter Sebastian Nordholt, Bernhard Haslhofer, and Matteo Maffei. Cross-layer deanonymization methods in the Lightning protocol. In *Financial Cryptography and Data Security (FC)*, 2021.
- [33] Jeremy Rubin. BIP-119: CHECKTEMPLATEVERIFY, 2020. <https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki>.
- [34] Thomas C. Schelling. *The Strategy of Conflict*. Harvard University Press, 1960.
- [35] Second. Lightning liquidity: Ark servers and LSPs compared, 2025. <https://second.tech/blog/ark-liquidity-research-01>, May 1, 2025.
- [36] Second. Forfeits & connectors, 2026. Documentation, <https://second.tech/docs/learn/forfeits>, accessed September 30, 2026.
- [37] Second. Ark VTXOs, 2026. Documentation, <https://second.tech/docs/learn/vtxo>, accessed September 30, 2026.
- [38] Gustavus J. Simmons. The prisoners’ problem and the subliminal channel. In *Advances in Cryptology: Proceedings of CRYPTO ’83*, 1984.
- [39] George J. Stigler. A theory of oligopoly. *Journal of Political Economy*, 72(1), 1964.
- [40] Sergei Tikhomirov, Rene Pickhardt, Alex Biryukov, and Mariusz Nowostawski. Probing channel balances in the Lightning Network. *arXiv preprint arXiv:2004.00333*, 2020.
- [41] Friedhelm Victor and Andrea Marie Weintraud. Detecting and quantifying wash trading on decentralized cryptocurrency exchanges. In *The Web Conference (WWW)*, 2021.

A Notation

Symbol	Meaning
$\rho = (\mathcal{A}, v, F, X, \Lambda, \Gamma, \tau)$	settlement object: authority, value, finality clock, exit, liveness, counterparty set, expiry
$\ell^U, \ell^C, \ell^V, \ell^E$	holdings by class
σ	settlement event (from-class, to-class, amount, clock condition)
O_A, I_A	cumulative outward, inward crossings of ∂A
S_A, S'_A, K_A, K'_A	outbound and inbound stock on ∂A ; stock added by capacity events
$\hat{f}_a, \hat{f}_S$	forecast inbound toward agent a ; forecast peak aggregate net receipt at server S
R_S, W_S, B_S	server liquidity stock, fronted-and-unrecovered value, total capital
L	mean residual lock on fronted value at forfeit
$\mathcal{D}_{\text{obj}}(D, H)$	liquidity duration: locked capital-time per unit delivered
$\mathcal{R}_{\text{cap}}, \mathcal{R}_{\text{svc}}$	unique-exposure and service relations
$\text{nf}, \text{cm}, \text{root}_n, k_\rho$	nullifier keyed by the object's authority key, claimant commitment, settled-set root
$E(B, \kappa, \Delta)$	escrow with bond B for connector κ and challenge period Δ
$O_{\text{chain}}, O_{\text{gossip}}, O_{\text{peer}}, O_{\text{server}}$	observer rows

B Reference model

The model accompanying this paper [12] is TypeScript with no dependencies, under `model/`, run with `node -experimental-strip-types -test "model/*.test.ts"`. It implements the extended ledger with clocked settlement events (Proposition 2); the server liquidity ledger under both recovery readings, the refresh cycle, the aggregate bound, and the server-tier term, with the channel model under oracle and trailing forecasts and the grid of Table 3 (Proposition 3); the epoched registry with authority-keyed nullifiers and attested successors (Propositions 4, 5); the escrow state machine with the published BitVM3 sizes as inputs (Proposition 6); and three warden statistics on a coalition fixture, with the two that fail asserted to fail. The proof system is not implemented; the registry verifies a simulated attestation with the interface a verifier would expose. Seventeen tests pass at the cited commit; they witness the propositions on finite cases and prove nothing about distributed execution.